



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Corso di Laurea in Informatica

Tesi di Laurea

CONFRONTO DI TECNICHE DI
STEGANOGRAFIA TRAMITE MODELLI
GENERATIVI

COMPARING OF STEGANOGRAPHY
TECHNIQUES USING GENERATIVE MODELS

MATTEO DICIOTTI

Relatore: Prof. *Daniele Baracchi*

Correlatrice: Dott.ssa *Giulia Bertazzini*

Correlatore: Prof. *Daniele Castellana*

Anno Accademico 2025-2026

INDICE

1	Introduzione	5
2	Fondamenti teorici e framework di riferimento	9
2.1	Steganografia: definizioni e principi	9
2.1.1	Il modello di comunicazione steganografica	9
2.1.2	Steganalisi e indistinguibilità statistica	10
2.1.3	Steganografia senza embedding (SwE)	11
2.1.4	Metriche e proprietà	11
2.1.5	Steganografia e watermarking: una distinzione formale	12
2.2	Il modello Meteor	13
2.2.1	Funzionamento generale	13
2.2.2	Arithmetic coding steganografico	14
2.2.3	Il canale testuale come canale privo di rumore	14
2.2.4	Garanzie di sicurezza	15
2.3	Architetture per la generazione visuale	15
2.3.1	Transformer	16
2.3.2	VQ-GAN	16
2.3.3	Open-MAGVIT2 e Lookup-Free Quantization (LFQ)	18
2.4	Glossario	20
3	Percorso sperimentale	21
3.1	Primo approccio: estensione di Meteor con VQ-GAN	21
3.1.1	Motivazione della scelta	21
3.1.2	Implementazione del sistema	22
3.1.3	Formalizzazione algoritmica	22
3.1.4	La non-invertibilità della VQ-GAN	24
3.1.5	Risultati e limitazioni	25
3.2	Tentativo di fine-tuning dell'encoder VQ-GAN	25
3.2.1	Obiettivo	25
3.2.2	Strategia di fine-tuning	25
3.2.3	Risultati	26
3.3	Equivalenza del Transformer: Meteor è applicabile ai token visivi	26
3.3.1	La verifica concettuale	26
3.3.2	Implicazione per la scelta dell'architettura	27
3.4	Sperimentazione con Open-MAGVIT2	27

3.4.1	Motivazione	27
3.4.2	Il problema della propagazione dell'errore	28
3.4.3	Misurazione dell'errore	29
3.4.4	Verifica: embedding nei piani di bit	30
3.4.5	Tensione fondamentale: robustezza vs impercettibilità	31
3.5	Sintesi dei risultati sperimentali	32
3.6	Limitazioni emerse	32
4	Conclusioni	35
4.1	Riepilogo dei risultati	35
4.2	Contributi	36
4.3	Lavori futuri	36
4.4	Considerazioni finali	39

"Questo è bello!"
"Eh, nella sua semplicità.."
"Questo è iper-realista."
"E se me lo concedi monocromatico."
"L'artista è Meteor."
"Meteor, già sentito!"
— Aldo, Giovanni e Giacomo

INTRODUZIONE

Garantire la confidenzialità, l'autenticità e l'integrità delle comunicazioni digitali è una delle esigenze fondamentali dell'architettura informatica contemporanea. Gran parte di questa garanzia è oggi affidata a primitive crittografiche e a protocolli consolidati come *Transport Layer Security* (TLS), il cui scopo è rendere il contenuto dei messaggi inaccessibile a chiunque non sia in possesso delle chiavi appropriate. La crittografia moderna, in altre parole, protegge *cosa* viene detto, ma non occulta il fatto che una comunicazione stia avvenendo.

Questa seconda proprietà, ovvero l'opacità del canale, è diventata cruciale in un numero crescente di scenari. La diffusione di meccanismi di sorveglianza di massa e di infrastrutture di censura, come quelle documentate rispetto al *Great Firewall* cinese [chinawall-2015] o relative al blocco di servizi di messaggistica in diversi contesti nazionali [conger-2016, sablah-2025], ha reso evidente che il traffico cifrato è facilmente identificabile e bloccabile da sistemi di filtraggio che operano a livello di rete. L'elevata entropia dei flussi TLS è, paradossalmente, un marcatore che li rende *visibili*. In scenari operativi ostili, quindi, la sola crittografia non è sufficiente: diventa necessario ricorrere a metodologie che rendano indistinguibile una comunicazione cifrata da un normale flusso di dati innocui.

È in questo contesto che si colloca la **steganografia**. La disciplina steganografica si dedica all'occultamento di informazioni all'interno di oggetti di copertura (*cover objects*) apparentemente ordinari, in modo che un osservatore esterno non possa nemmeno stabilire se una comunicazione nasconda informazioni al suo interno. A differenza della crittografia, la cui sicurezza poggia sulla complessità computazionale dei problemi matematici sottostanti, la sicurezza steganografica si fonda sull'**indistinguibilità statistica** [turch-2020]: un sistema steganografico è tanto più sicuro quanto più la distribuzione dei propri *stego-object* è simile a quella degli oggetti naturali prodotti dal medesimo processo

comunicativo.

Le tecniche steganografiche classiche, come la manipolazione dei bit meno significativi (*Least Significant Bit*, **LSB**) nelle immagini, si sono rivelate progressivamente vulnerabili all'evoluzione delle tecniche di steganalisi, che combinano analisi statistica e apprendimento automatico per rilevare perturbazioni anche minime [apau-2024, subramanian-2021]. La ricerca recente ha quindi intrapreso un cambio di paradigma: anziché modificare un contenuto esistente per nascondervi un messaggio, si sfruttano **modelli generativi** basati su reti neurali profonde per *progettare ex novo* il contenuto stesso in funzione del messaggio da occultare. Questo approccio, indicato in letteratura come *Steganography without Embedding* (**SwE**) [liu-2020, dong-2025], sposta la steganografia dal piano della perturbazione impercettibile a quello della generazione indistinguibile.

In questo filone si inserisce **Meteor** [kaptchuk-2021], un algoritmo di steganografia a chiave simmetrica pensato per il dominio testuale. Meteor sfrutta un *Large Language Model* (**LLM**) per creare un canale di comunicazione occulto: anziché modificare un testo esistente, *guida* il processo di generazione del modello, pilotando la scelta dei token successivi in base ai bit del messaggio segreto. La sicurezza del sistema poggia sulla garanzia formale che lo stego-text così prodotto sia statisticamente indistinguibile dall'output naturale del modello. Meteor costituisce, a oggi, una delle realizzazioni più convincenti del paradigma SwE.

La presente tesi si propone di indagare se un approccio concettualmente analogo a Meteor possa essere applicato al dominio visivo delle immagini. L'obiettivo non è implementare un sistema pronto all'uso, ma esplorare sistematicamente le condizioni sotto le quali un modello generativo visuale può essere impiegato come sampler steganografico, identificando i fattori strutturali che lo rendono possibile o, come si vedrà, lo limitano. L'indagine ruota attorno a due aspetti complementari: da un lato la scelta del tokenizer visivo, che determina la granularità della rappresentazione e la fedeltà del round-trip tra *spazio latente* dei token e spazio dei pixel; dall'altro la *natura autoregressiva* dei moderni generatori visivi, che introduce una dipendenza sequenziale tra token consecutivi e condiziona il modo in cui gli errori di trasmissione possono propagarsi.

Il percorso sperimentale, descritto in dettaglio nel [Capitolo 3](#), si articola in tappe successive. Si parte da **VQ-GAN** [esser-2020], un'architettura rappresentativa della prima generazione di tokenizer visivi, per poi spostarsi su **Open-MAGVIT2** [luo-2025], che introduce una forma di quantizzazione (*Lookup-Free Quantization*, **LFQ**) con proprietà strutturalmente diverse. Per ciascuna architettura, la sperimentazione valuta la

fattibilità di guidare il campionamento autoregressivo come accade in Meteor, ricercando le cause che impediscono, allo stato attuale, l'applicazione di tale approccio al dominio visivo.

Il presente elaborato è organizzato come segue:

- Il **Capitolo 2** introduce i concetti teorici necessari: il modello di comunicazione steganografica, la nozione di indistinguibilità statistica, il paradigma SwE, il framework Meteor e le architetture di tokenizzazione visiva (VQ-GAN e Open-MAGVIT2 con LFQ) impiegate nella sperimentazione.
- Il **Capitolo 3** costituisce il nucleo del lavoro: descrive il percorso sperimentale seguito, i problemi incontrati, le soluzioni parziali esplorate e i risultati ottenuti. In particolare, viene analizzata la non-invertibilità della VQ-GAN, l'equivalenza concettuale dell'algoritmo di Meteor a livello di Transformer, il problema della propagazione dell'errore nei modelli autoregressivi e la strategia di embedding nei piani di bit.
- Il **Capitolo 4** sintetizza i contributi della tesi, discute le limitazioni emerse e individua possibili direzioni di sviluppo futuro.

FONDAMENTI TEORICI E FRAMEWORK DI RIFERIMENTO

Questo capitolo introduce i concetti fondamentali, le definizioni e i framework necessari per comprendere il percorso sperimentale descritto nei capitoli successivi. L'esposizione procede per livelli di astrazione crescente: dalla steganografia classica agli algoritmi generativi, fino agli specifici modelli architetturali impiegati nella sperimentazione.

2.1 STEGANOGRAFIA: DEFINIZIONI E PRINCIPI

2.1.1 *Il modello di comunicazione steganografica*

La steganografia è la disciplina che si occupa di occultare l'esistenza stessa di una comunicazione all'interno di un oggetto multimediale apparentemente innocuo [turch-2020]. A differenza della crittografia, che protegge il contenuto del messaggio rendendolo inaccessibile, la steganografia nasconde il fatto stesso che una comunicazione stia avvenendo.

Il modello classico di comunicazione steganografica coinvolge i seguenti attori e oggetti:

MITTENTE: intende inviare un messaggio segreto M al destinatario senza che un attore intermedio (un *ISP* per esempio), o *Avversario*, possa rilevare la presenza della comunicazione.

DESTINATARIO: riceve e decodifica il messaggio nascosto.

AVVERSAIO: osserva il canale di comunicazione e tenta di determinare se un messaggio nascosto sia presente (steganalisi).

COVER-OBJECT : oggetto multimediale originale, nella sua forma inalterata, che funge da contenitore all'interno del quale si intende occultare un messaggio segreto.

STEGO-OBJECT : oggetto risultante dall'incorporazione del messaggio segreto all'interno del cover-object. La proprietà fondamentale che tale artefatto deve soddisfare è l'indistinguibilità percettiva e statistica dal cover-object originale [simmons-1984]. Nel caso di *Meteor* o delle VQ-GAN, il termine si riferisce all'oggetto **generato** che contiene l'informazione segreta codificata, che non è in senso stretto un "oggetto modificato" ma un "oggetto creato" ad hoc. In questo contesto, il termine "stego-object" è usato in modo estensivo per indicare qualsiasi output generato che contenga un messaggio segreto, indipendentemente dal fatto che sia stato ottenuto tramite embedding o generazione ex novo.

Nel contesto della *Steganography without Embedding (SwE)*, quindi, i termini assumono un'accezione estesa: poiché lo stego-object non è un cover-object modificato in senso stretto, sebbene come vedremo con VQ-GAN alcune architetture generino immagini a partire da altre immagini, ma è un oggetto **generato** a partire dal contesto, che può comprendere un'immagine nota, e un messaggio segreto. L'indistinguibilità è garantita dalla qualità del modello generativo, che deve produrre output statisticamente equivalenti a campioni naturali o in alternativa a prodotti tipici del medesimo processo generativo sottratto il messaggio segreto. Quest'ultimo caso si applica in particolare nei contesti nei quali i prodotti di modelli generativi risultano fruiti dagli utenti finali e quindi plausibilmente trasmessi attraverso canali di comunicazione, come sempre maggiormente accade con i modelli di generazione di immagini e video.

2.1.2 *Steganalisi e indistinguibilità statistica*

La steganalisi è la disciplina deputata al rilevamento di messaggi occulti all'interno di oggetti multimediali, avvalendosi di metodologie di analisi statistica e apprendimento automatico [apau-2024]. La steganalisi moderna impiega tecniche sempre più sofisticate per esaminare le proprietà statistiche degli oggetti multimediali.

La nozione di **perfectly secure steganography** [kaptchuk-2021] definisce le condizioni per cui un sistema steganografico è sicuro: uno stego-object è perfettamente sicuro se la sua distribuzione di probabilità è identica a quella dei cover-object naturali prodotti dal medesimo processo generativo. Formalmente, sia $\mathcal{G}$ un modello generativo che produce campioni dalla distribuzione $P_{\mathcal{G}}$, e sia $\mathcal{S}$ un sistema steganografico che, dati un messaggio M e una chiave segreta K , produce uno stego-object x .

Sia $P_S(\mathbf{x} \mid M)$ la distribuzione di probabilità degli stego-object generati da S per il messaggio M , marginalizzata su tutte le possibili chiavi K (la cui scelta casuale fornisce l'entropia necessaria a garantire l'indistinguibilità). Il sistema è perfettamente sicuro se:

$$\forall M, \quad P_G(\mathbf{x}) = P_S(\mathbf{x} \mid M).$$

In altre parole, per qualsiasi messaggio M , un osservatore che esamina $\mathbf{x}$ non può determinare se esso provenga dal modello generativo G o dal sistema steganografico S , poiché le due distribuzioni sono indistinguibili.

2.1.3 *Steganografia senza embedding (SwE)*

La *Steganography without Embedding (SwE)* rappresenta un cambio di paradigma rispetto alla steganografia classica: invece di modificare un cover-object preesistente inserendoci un messaggio segreto all'interno (*embedding*), il cover-object stesso viene *generato ex novo* a partire dal messaggio segreto. Un modello generativo produce un artefatto multimediale la cui struttura è determinata dai bit del messaggio da occultare.

Poiché non esiste un cover-object originale di riferimento, la steganalisi classica basata sul confronto tra cover e stego risulta inefficace. L'indistinguibilità è garantita esclusivamente dalla qualità del modello generativo. Paradigmi come **Meteor** per il dominio testuale e la presente sperimentazione per il dominio visivo rappresentano istanze di questo approccio, sebbene permanga la necessità di un'immagine preesistente come contesto per la generazione, che può essere utilizzata, in certa misura, per il confronto statistico steganalitico.

2.1.4 *Metriche e proprietà*

Nel corso della trattazione verranno utilizzati i seguenti concetti quantitativi:

- **Payload:** quantità di informazione segreta, espressa in bit o byte, che può essere occultata all'interno di un singolo oggetto multimediale senza comprometterne l'integrità statistica.
- **Bitrate variabile:** strategia di codifica in cui la quantità di informazione incorporata in ciascun token può variare dinamicamente in funzione delle caratteristiche del contesto e della distribuzione di probabilità. In contesti a bassa entropia il sistema può astenersi

dalla codifica, mentre in scenari ad alta entropia può codificare un numero maggiore di bit per token.

- **Token flip rate:** frazione di token che subiscono una modifica (un *flip*) durante un processo di ricodifica. Nel contesto della presente sperimentazione, misura la proporzione di token visivi che non sopravvivono al passaggio encoding → rendering → decoding.
- **Per-bit flip rate:** probabilità che un singolo bit (o dimensione latente) venga alterato durante il passaggio attraverso un canale rumoroso.

2.1.5 *Steganografia e watermarking: una distinzione formale*

Sebbene steganografia e watermarking condividano la tecnica di incorporare informazione all'interno di un oggetto multimediale, le due discipline si distinguono per l'obiettivo di sicurezza e per il modello di avversario, con conseguenze dirette sui requisiti progettuali e sulle metriche di valutazione.

Nella **steganografia**, l'obiettivo primario è l'*indistinguibilità*: lo stego-object deve essere statisticamente indistinguibile dai cover-object naturali, poiché l'avversario (lo steganalista) tenta di rilevare la presenza stessa del messaggio nascosto. La metrica di sicurezza fondamentale è la divergenza tra la distribuzione degli stego-object e quella dei cover-object: minore è questa divergenza, maggiore è la sicurezza. La capacità di payload è un obiettivo secondario, subordinato al mantenimento dell'indistinguibilità.

Nel **watermarking**, l'obiettivo primario è la *robustezza*: il segnale incorporato (il watermark) deve sopravvivere a trasformazioni ostili deliberate (compressione, ridimensionamento, ritaglio, filtraggio) che un attaccante potrebbe applicare per rimuoverlo. L'avversario non cerca di rilevare la presenza del watermark, che è tipicamente nota o irrilevante, ma di distruggerlo senza degradare eccessivamente l'oggetto. La metrica fondamentale è il *bit error rate* (BER) del watermark recuperato dopo le trasformazioni avversarie. L'impercettibilità è un vincolo, non l'obiettivo: il watermark deve essere invisibile all'osservatore umano, ma può lasciare tracce statistiche rilevabili.

La distinzione può essere formalizzata come segue. Sia $\mathbf{x}$ il cover-object, M il messaggio da incorporare e $\mathbf{y}$ lo stego-object (o oggetto watermarked). In steganografia, il requisito è:

$$D(P(\mathbf{y} | M), P(\mathbf{x})) < \varepsilon,$$

dove D è una misura di divergenza statistica (ad esempio la divergenza di Kullback-Leibler) e ε è sufficientemente piccolo da rendere la rilevazione computazionalmente non praticabile. Nel watermarking, il requisito è:

$$\forall T \in \mathcal{T}, \quad \text{BER}(M, \text{Extract}(T(y))) < \delta,$$

dove $\mathcal{T}$ è l'insieme delle trasformazioni avversarie considerate, Extract è la funzione di estrazione del watermark, e δ è la soglia di errore tollerabile.

Questa distinzione è centrale per comprendere i risultati della sperimentazione descritta nel [Capitolo 3](#): le strategie basate sul campionamento autoregressivo del Transformer (Meteor-like) realizzano una steganografia statisticamente indistinguibile ma fragile al rumore del canale, mentre le strategie basate sulla modulazione diretta dei codici LFQ (QIM) realizzano un watermarking robusto ma statisticamente rilevabile. La tensione tra questi due obiettivi, indistinguibilità e robustezza, emerge come il trade-off fondamentale del dominio.

2.2 IL MODELLO METEOR

Meteor [kaptchuk-2021] è un framework steganografico a chiave simmetrica che realizza il paradigma SwE nel dominio testuale utilizzando un modello linguistico autoregressivo (GPT-2). Il principio fondante è il seguente: poiché un LLM genera testo campionando da una distribuzione di probabilità sui token successivi, è possibile *guidare deterministicamente* questa selezione utilizzando i bit del messaggio segreto, piuttosto che campionare casualmente.

2.2.1 Funzionamento generale

Il protocollo si articola nelle seguenti fasi:

1. **Codifica steganografica:** dato un messaggio segreto M e una chiave K , il sistema genera un testo T (lo *stego-text*) in cui il messaggio è nascosto. La generazione avviene passo-passo: a ciascun passo temporale t , il modello produce una distribuzione di probabilità $\mathbf{p}_t = P(\cdot \mid \mathbf{x}_{<t})$ sull'intero vocabolario. I bit del messaggio determinano la selezione del token x_t tramite un meccanismo di *arithmetic coding* sul prefisso della distribuzione troncata.

2. **Decodifica steganografica:** il destinatario, in possesso dello stesso modello e della stessa chiave, ripercorre la sequenza di token. Per ciascuna posizione, ricostruisce la medesima distribuzione di probabilità e, confrontando il token effettivamente presente nel testo ricevuto con la distribuzione attesa, recupera i bit del messaggio originale.

2.2.2 *Arithmetic coding steganografico*

Il campionamento steganografico dei token in *Meteor* è realizzato mediante *arithmetic coding*, la tecnica di codifica entropica su cui l'intero framework si fonda.

Il messaggio segreto, una volta cifrato, viene rappresentato come una sequenza binaria. A ciascun passo di generazione t , il modello produce una distribuzione di probabilità sul vocabolario. I token con probabilità inferiore a una soglia prefissata vengono esclusi per evitare la selezione forzata di token improbabili, e le probabilità dei token rimanenti vengono rinormalizzate. Fissata una precisione di D bit, l'intervallo discreto $[0, 2^D)$ viene ripartito in sotto-intervalli cumulativi proporzionali alle probabilità così ottenute, associando a ciascun token superstite un intervallo $[C_{i-1}, C_i)$. Si preleva quindi dalla testa del messaggio un blocco di D bit, lo si interpreta come intero $m \in [0, 2^D)$, e si seleziona il token i^* il cui intervallo cumulativo contiene m , ovvero tale che $C_{i^*-1} \leq m < C_{i^*}$. Il numero di bit effettivamente codificati da questo token è dato dalla lunghezza del prefisso binario comune tra l'estremo sinistro C_{i^*-1} e l'estremo destro C_{i^*} : tali bit vengono consumati, cioè rimossi dalla testa della sequenza del messaggio, e il procedimento si ripete al passo successivo.

La decodifica è simmetrica: data la stessa distribuzione e l'indice del token i^* presente nello stego-text, si determina l'intervallo $[C_{i^*-1}, C_{i^*})$ e si estrae il prefisso binario comune.

2.2.3 *Il canale testuale come canale privo di rumore*

Meteor, e più in generale qualsiasi schema di SwE applicato al dominio testuale, beneficia di una proprietà che può apparire ovvia ma che è invece decisiva quando si tenta la trasposizione ad altri domini mediati: il testo, nella sua rappresentazione digitale, costituisce un *canale di comunicazione intrinsecamente privo di rumore*. I byte che rappresentano le frasi generate vengono trasmessi in maniera precisa: non subiscono

compressione con perdita di informazione, né quantizzazione, né alcun altro tipo di degradazione. Di conseguenza, il token x_t selezionato dal mittente durante la codifica steganografica è *esattamente* il token che il destinatario legge durante la decodifica.

Questa proprietà è il fondamento silenzioso su cui poggia l'intero meccanismo di Meteor. L'arithmetic coding descritto nella sezione precedente presuppone che la sequenza di token su cui il mittente si è condizionato sia la stessa che il destinatario osserverà, condizione che nel dominio testuale è sempre verificata. Nel dominio visivo, al contrario, il passaggio attraverso i pixel introduce inevitabilmente una forma di rumore di canale: la quantizzazione a 8 bit per canale colore, la compressione (anche in formati considerati *lossless* come il PNG, che quantizzano comunque a uint8) e, più in generale, qualsiasi trasformazione dell'immagine alterano i token visivi tra il momento in cui vengono generati e il momento in cui vengono ricodificati dal destinatario. Come si vedrà nel [Capitolo 3](#), questa differenza fondamentale — un canale testuale *lossless* contrapposto a un canale visivo inevitabilmente *lossy* — è la ragione ultima per cui Meteor funziona con affidabilità completa nel dominio per cui è stato progettato, e al contempo la causa primaria delle difficoltà che si incontrano nel trasporlo alle immagini.

2.2.4 Garanzie di sicurezza

La sicurezza di Meteor poggia su due proprietà fondamentali:

- **Indistinguibilità statistica:** i token vengono selezionati dalla stessa distribuzione di probabilità del modello, quindi la distribuzione degli stego-text coincide con quella dei testi naturali generati dal modello.
- **Soglia di entropia:** il sistema si astiene dalla codifica in contesti a bassa entropia (dove la distribuzione è molto concentrata su pochi token), prevenendo la selezione forzata di token improbabili.

2.3 ARCHITETTURE PER LA GENERAZIONE VISUALE

Questa sezione introduce le architetture di deep learning utilizzate nel percorso sperimentale per la generazione e la tokenizzazione delle immagini.

2.3.1 Transformer

L'architettura Transformer [vaswani-2017] costituisce il fondamento dei moderni modelli generativi sia nel dominio testuale che in quello visivo. Il suo componente chiave è il meccanismo di *self-attention*, che permette a ciascuna posizione di una sequenza di aggregare informazioni da tutte le altre posizioni con pesi determinati dinamicamente.

Nel contesto della generazione di immagini, il Transformer viene impiegato per modellare la sequenza di token visivi in modo autoregressivo. Data una sequenza di ℓ token $\mathbf{s} = (s_1, \dots, s_\ell)$, l'architettura fattorizza la probabilità congiunta come:

$$P(\mathbf{s}) = \prod_{t=1}^{\ell} P(s_t | s_{<t}).$$

La generazione procede quindi iterativamente: a ogni passo t , il modello produce una distribuzione di probabilità sul prossimo token s_t condizionata dai token già generati $s_{<t}$.

2.3.2 VQ-GAN

La *Vector Quantized-Generative Adversarial Network* (VQ-GAN) [esser-2020] è un'architettura che integra quantizzazione vettoriale, un autoencoder convoluzionale e una loss avversaria per la generazione di immagini di alta qualità. La VQ-GAN si compone di due stadi:

Primo stadio: tokenizer convoluzionale

Il primo stadio è un autoencoder convoluzionale che trasforma un'immagine in una griglia di token discreti e la ricostruisce. I componenti principali sono:

- **Encoder convoluzionale** E : riduce la risoluzione spaziale dell'immagine $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ mediante strati convoluzionali con striding, producendo una mappa di feature $\hat{\mathbf{z}} \in \mathbb{R}^{h \times w \times n_z}$, dove $h = H/f$, $w = W/f$ e f è il fattore di compressione spaziale (tipicamente $f = 16$).
- **Codebook** $\mathcal{K} = \{\mathbf{e}_1, \dots, \mathbf{e}_K\}$: un insieme di K vettori latenti appresi (tipicamente $K = 2^{14}$). Ciascuna posizione (i, j) della mappa latente

$\hat{\mathbf{z}}$ viene quantizzata al vettore del codebook più vicino in norma euclidea:

$$\mathbf{z}_q^{(i,j)} = \mathbf{e}_{k^*}, \quad k^* = \arg \min_k \|\hat{\mathbf{z}}^{(i,j)} - \mathbf{e}_k\|_2.$$

L'immagine viene quindi rappresentata come una griglia di indici $\mathbf{S} \in \mathcal{K}^{h \times w}$.

- **Decoder convoluzionale G:** trasforma la griglia di vettori quantizzati $\mathbf{z}_q$ nuovamente nello spazio immagine, tramite strati di transposed convolution che effettuano upsampling fino alla risoluzione originale.

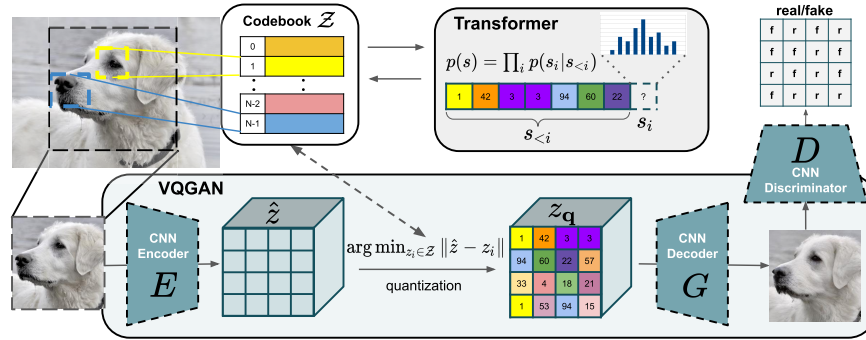


Figura 1: Rappresentazione schematica dell'architettura VQ-GAN: l'encoder convoluzionale riduce l'immagine in una griglia di token discreti tramite quantizzazione vettoriale sul codebook; il Transformer autoregressivo modella la distribuzione dei token; il decoder convoluzionale ricostruisce l'immagine. Fonte: [esser-2020].

Funzione di loss e addestramento

La VQ-GAN introduce due innovazioni rispetto alla VQ-VAE [van-den-oord-2017], sua precorritrice:

- **Loss avversaria:** un discriminatore PatchGAN [isola-2017] viene addestrato a distinguere immagini reali da ricostruzioni, mentre il decoder cerca di ingannarlo. Questo produce ricostruzioni più nitide, specialmente nelle texture ad alta frequenza.
- **Loss percettiva:** le attivazioni di una rete VGG-16 [simonyan-2015] pre-addestrata vengono confrontate tra immagine originale e ricostruita a vari livelli di astrazione, penalizzando deviazioni semantiche.

Secondo stadio: Transformer autoregressivo

Il secondo stadio è un modello Transformer autoregressivo che apprende la distribuzione dei token visivi sulla griglia. Dato un contesto di $p \times p$ token, il modello prevede la distribuzione di probabilità sul token successivo. Questo stadio opera esclusivamente sugli indici discreti del codebook e non interagisce direttamente con lo spazio immagine.

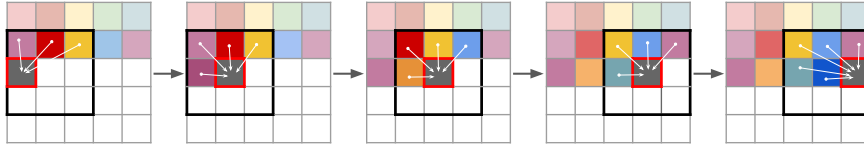


Figura 2: Meccanismo di attention del Transformer autoregressivo nella VQ-GAN: la generazione del token successivo è condizionata dai token circostanti nella griglia spaziale, permettendo al modello di catturare dipendenze locali e globali. Fonte: [esser-2020].

2.3.3 Open-MAGVIT2 e Lookup-Free Quantization (LFQ)

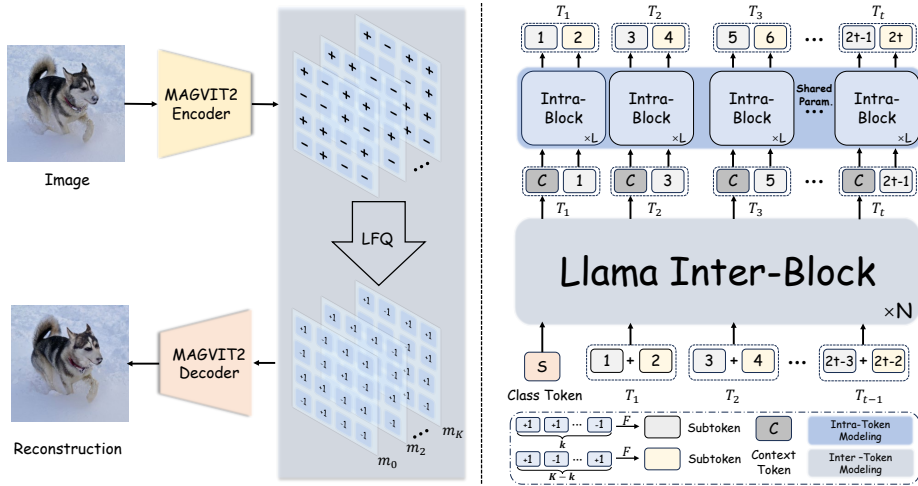


Figura 3: Architettura e funzionamento di Open-MAGVIT2: il framework combina un encoder convoluzionale con Lookup-Free Quantization (LFQ) per produrre token binari senza codebook esplicito, e un Transformer autoregressivo (Net2NetTransformer) per la generazione della sequenza di token. Fonte: [luo-2025].

Open-MAGVIT2 [luo-2025] è un tokenizer visivo all'avanguardia che introduce la *Lookup-Free Quantization (LFQ)*, una tecnica di quantizzazione

che elimina la necessità di un codebook esplicito.

Principio della LFQ

A differenza della VQ-GAN, dove ciascun vettore latente $\hat{\mathbf{z}}^{(i,j)} \in \mathbb{R}^{n_z}$ viene confrontato per similarità con tutti i K vettori del codebook — un’operazione di costo $\mathcal{O}(K \cdot n_z)$ per posizione spaziale — la LFQ quantizza ciascuna dimensione latente in modo indipendente, eliminando del tutto il codebook. Un’immagine viene processata da un encoder convoluzionale che produce, per ciascuna posizione spaziale, un vettore di d scalari latenti $\hat{\mathbf{z}}^{(i,j)} \in \mathbb{R}^d$. Ciascuno scalare viene quantizzato al proprio **segno** ($+1$ o -1), producendo un codice binario di d bit per posizione spaziale. Formalmente:

$$\mathbf{z}_q^{(i,j)}[k] = \text{sign}\left(\hat{\mathbf{z}}^{(i,j)}[k]\right) \in \{-1, +1\}, \quad k = 1, \dots, d.$$

L’insieme dei possibili codici è quindi lo spazio $\{-1, +1\}^d$, che in Open-MAGVIT2 ha dimensione $d = 18$ bit, per un totale implicito di $2^{18} = 262\,144$ possibili codici. Non essendo necessaria una ricerca nel codebook, la quantizzazione ha costo $\mathcal{O}(d)$ per posizione, da cui il nome “lookup-free”.

Fattorizzazione asimmetrica

In Open-MAGVIT2, i 18 bit vengono suddivisi in due sottotoken:

- **Pre-token** (pre): 6 bit (bit 0–5, corrispondenti a $2^6 = 64$ possibili valori).
- **Post-token** (post): 12 bit (bit 6–17, corrispondenti a $2^{12} = 4096$ possibili valori).

Il modello autoregressivo (*Net2NetTransformer*) predice prima il pre, poi il post condizionato dal pre. Questa fattorizzazione asimmetrica [6, 12] è motivata da considerazioni di efficienza computazionale: lo spazio di ricerca del primo sottotoken è ridotto (64 opzioni), mentre il secondo sottotoken ha maggiore capacità espressiva.

Un aspetto architetturale importante è lo **ShiftPermuter**, un modulo che riordina la sequenza di token tra la disposizione raster (righe della griglia) e l'ordine di generazione del Transformer. È cruciale utilizzare le funzioni `encode_to_z` e `decode_to_img` fornite dal modello, che applicano e rimuovono automaticamente la permutazione. Un uso diretto delle funzioni di encoder/decoder senza passare attraverso queste astrazioni produce una griglia con i token disposti in un ordine scorretto (permutati in modo incoerente rispetto all'originale) e un flip rate dei token prossimo al 100%.

Invertibilità della LFQ

La LFQ è intrinsecamente invertibile: poiché il codice è determinato unicamente dal segno di ciascuna dimensione latente, e la decodifica è una semplice mappa dal codice binario allo spazio immagine, non esiste ambiguità nel processo di quantizzazione. Questo rappresenta un vantaggio fondamentale rispetto alla VQ-GAN, dove la quantizzazione tramite codebook introduce una perdita informativa.

2.4 GLOSSARIO

Nel corso della trattazione vengono impiegati i seguenti termini tecnici:

- **Logits:** valori numerici prodotti dall'ultimo strato di un modello generativo prima dell'applicazione della funzione softmax per ottenere la distribuzione di probabilità.
- **Fine tuning:** addestramento supplementare di un modello pre-addestrato su un dataset o un obiettivo specifico, con tasso di apprendimento ridotto per preservare le conoscenze acquisite.
- **QIM (Quantization Index Modulation):** tecnica di watermarking che modula l'indice di quantizzazione di un segnale per incorporare informazione. Nel contesto della LFQ, consiste nel forzare il segno di dimensioni latenti selezionate per codificare i bit del messaggio.
- **Propagazione dell'errore (*error cascade*):** fenomeno per cui un singolo errore in una sequenza generata autoregressivamente corrompe tutte le predizioni successive, poiché ogni token x_t dipende da tutti i token precedenti $x_{<t}$.

PERCORSO SPERIMENTALE

Questo capitolo descrive il percorso di sperimentazione condotto per verificare l'applicabilità del paradigma steganografico *Meteor* al dominio visivo. L'esposizione segue l'ordine cronologico degli esperimenti, evidenziando le problematiche emerse e le strategie adottate per affrontarle. Il capitolo si conclude con un'analisi critica dei risultati, che identifica nella propagazione dell'errore dovuta all'autoregressività dei modelli utilizzati il fattore limitante principale per la realizzazione di uno SwE visivo basato su tokenizer convoluzionali.

3.1 PRIMO APPROCCIO: ESTENSIONE DI METEOR CON VQ-GAN

3.1.1 *Motivazione della scelta*

L'architettura VQ-GAN [esser-2020] è stata selezionata come primo candidato per estendere Meteor al dominio visivo perché offre un'analogia strutturale quasi perfetta con la pipeline testuale. Il suo encoder convoluzionale quantizza la rappresentazione latente in un codebook di K vettori discreti (tipicamente $K = 2^{14}$), producendo una griglia di indici interi $\mathbf{S} \in \mathcal{K}^{h \times w}$ che costituisce l'equivalente visivo della sequenza di token su cui opera un modello linguistico. Il secondo stadio della VQ-GAN è un Transformer autoregressivo addestrato a modellare la distribuzione $P(s_t | s_{<t})$ proprio su quegli indici: a ogni passo genera una distribuzione di probabilità sull'intero codebook, esattamente come un LLM produce una distribuzione sul vocabolario. Questa corrispondenza suggeriva che l'algoritmo di arithmetic coding steganografico di Meteor ([sottosezione 2.2.2](#)) potesse essere trasposto in modo immediato, sostituendo il vocabolario testuale con il codebook della VQ-GAN.

3.1.2 Implementazione del sistema

Il sistema implementato segue il seguente flusso operativo:

1. **Preparazione del contesto:** viene selezionata un'immagine dal dataset di riferimento (il modello S-FLCKR, addestrato su *outdoor landscapes* di *Taming Transformers* [esser-2020]). L'immagine viene processata dall'encoder VQ-GAN per ottenere una sequenza di indici discreti del codebook.
2. **Generazione autoregressiva vincolata:** per ciascuna posizione della griglia, il Transformer produce una distribuzione di probabilità su tutti i K codici del codebook. Il messaggio segreto determina la selezione del token tramite arithmetic coding ([sottosezione 2.2.2](#)), analogamente a quanto avviene in Meteor.
3. **Ricostruzione dell'immagine:** la sequenza completa di indici viene decodificata dal decoder VQ-GAN per produrre l'immagine finale (*stego-image*).
4. **Estrazione del messaggio:** il destinatario, in possesso dello stesso modello, ripercorre la sequenza di token, ricostruisce le distribuzioni di probabilità e recupera i bit del messaggio originale.

3.1.3 Formalizzazione algoritmica

La presente sezione formalizza, tramite pseudocodice e notazione matematica, gli algoritmi discussi nel flusso operativo sopra descritto. Il simbolo $|$ indica la concatenazione di stringhe e $\lfloor \cdot \rfloor$ l'arrotondamento all'intero più vicino.

Selezione steganografica del token

L'[Algoritmo 1](#) implementa il meccanismo di *arithmetic coding* che costituisce il cuore della codifica: dato il contesto corrente, il Transformer produce una distribuzione di probabilità sui K codici del codebook; i token con probabilità inferiore a $1/K$ vengono scartati, le probabilità residue vengono rinormalizzate e quantizzate a D bit di precisione, e il messaggio in ingresso determina quale token selezionare. Il numero di bit consumati è pari alla lunghezza del prefisso binario comune tra gli estremi dell'intervallo cumulativo associato al token scelto.

Algoritmo 1 Selezione steganografica del token visivo

Require: Contesto $\mathbf{c}$, buffer di messaggio $\mathbf{b} \in \{0,1\}^*$, dimensione codebook K , precisione D , soglia T

Ensure: Indice selezionato i^* , prefisso binario $\mathbf{b}_{\text{out}}$

- 1: $\mathbf{p} \leftarrow \text{Softmax}(\text{Transformer}(\mathbf{c}))$
- 2: $\mathcal{J} \leftarrow \{i \mid p_i \geq 1/K\}$ $\triangleright$ soppressione delle code a bassa probabilità
- 3: $\mathcal{J} \leftarrow \mathcal{J}[0 : \min(|\mathcal{J}|, T)]$ $\triangleright$ troncamento al top- T
- 4: **for all** $i \in \mathcal{J}$ **do**
- 5: $\hat{p}_i \leftarrow \lfloor (p_i / \sum_{j \in \mathcal{J}} p_j) \cdot 2^D \rfloor$ $\triangleright$ quantizzazione a D bit
- 6: **end for**
- 7: $C_0 \leftarrow 0$
- 8: **for** $k = 1$ **to** $|\mathcal{J}|$ **do**
- 9: $C_k \leftarrow C_{k-1} + \hat{p}_{\mathcal{J}[k-1]}$ $\triangleright$ cumulativa discreta
- 10: **end for**
- 11: $m \leftarrow \text{BinToInt}(\mathbf{b}[0 : D - 1])$ $\triangleright$ interpreta i primi D bit come intero
- 12: $i^* \leftarrow \mathcal{J}[\min\{k \mid C_k > m\}]$
- 13: $L \leftarrow C_{i^*-1}, R \leftarrow C_{i^*}$
- 14: $\mathbf{b}_{\text{out}} \leftarrow \text{CommonPrefix}(\text{IntToBin}(L, D), \text{IntToBin}(R - 1, D))$
- 15: **return** $(i^*, \mathbf{b}_{\text{out}})$

A valle della selezione del token, il sistema procede posizione per posizione lungo la griglia: per ciascuna posizione viene costruito il contesto locale a partire dalla griglia di riferimento $\mathbf{R}$ e dalla griglia in costruzione $\mathbf{B}$, si invoca la procedura di selezione steganografica, e si assegna il token scelto alla griglia. L'operazione si ripete per tutte le posizioni residue; esaurito il messaggio, le posizioni rimanenti vengono completate con campionamento non vincolato dalla distribuzione del Transformer, così da preservare la coerenza visiva della regione generata.

La decodifica opera in modo simmetrico: il destinatario ricostruisce il medesimo contesto iniziale, codifica l'immagine stego ricevuta nella griglia di indici tramite l'encoder VQ-GAN, e percorre le posizioni residue ricostruendo la distribuzione troncata e quantizzata a partire dal contesto corrente; individua quindi la posizione del token ricevuto nell'insieme selezionato e recupera il prefisso binario comune degli estremi dell'intervallo cumulativo corrispondente, concatenando progressivamente i bit estratti fino a ricostruire l'intero messaggio.

3.1.4 La non-invertibilità della VQ-GAN

Durante i test preliminari è emersa una criticità fondamentale: l'encoder e il decoder della VQ-GAN non costituiscono funzioni inverse esatte. Formalmente:

$$\text{Decoder}_{VQ}(\text{Encoder}_{VQ}(\mathbf{x})) \neq \mathbf{x}, \quad \text{per molti } \mathbf{x} \in \mathbb{R}^{H \times W \times C}.$$

Circa il 19% dei token generati dal Transformer, decodificati tramite il decoder VQ-GAN e ricodificati utilizzando l'encoder, non corrispondono. Questa asimmetria implica che il token selezionato durante l'encoding steganografico potrebbe non corrispondere a quello ottenuto ricodificando l'immagine generata tramite l'encoder VQ-GAN. In altre parole, dopo aver scelto un indice i^* per una posizione (r, c) , aver decodificato l'immagine e averla riprocessata con l'encoder, la posizione (r, c) potrebbe produrre un indice diverso $i' \neq i^*$.

Il problema è strutturale: la quantizzazione vettoriale, la loss avversaria e la loss percettiva addestrano l'autoencoder a produrre ricostruzioni visivamente fedeli, non a garantire l'invertibilità del codice. La mappa dallo spazio immagine allo spazio dei codici discreti non è iniettiva, e codici diversi possono produrre pixel simili (e viceversa).

Abbiamo quindi esplorato la possibilità di introdurre un meccanismo di verifica che, dopo aver selezionato un token i^* in una posizione (r, c) , decodificasse l'intera griglia di indici fino a quel momento costruita, la ricodificasse con l'encoder, e verificasse che il token nella posizione (r, c) corrispondesse effettivamente a i^* . L'idea era iterare la selezione fino a trovare un token che sopravvivesse al round-trip encoding-decoding-encoding. Tuttavia, questa strategia si è rivelata impraticabile per una ragione architettonica di fondo: il decoder VQ-GAN è composto da strati convoluzionali che operano sull'intera griglia spaziale, e la decodifica di una griglia parzialmente popolata, in cui molte posizioni contengono ancora un valore di default o non sono ancora state generate, produce un'immagine che non corrisponde a una porzione dell'immagine finale. In altri termini, la decodifica parziale non è equivalente alla decodifica completa ristretta a una sotto-regione: i filtri convoluzionali mescolano informazione attraverso i confini delle posizioni già fissate e di quelle ancora indefinite, invalidando la verifica locale. Il meccanismo è stato pertanto accantonato.

3.1.5 Risultati e limitazioni

L'impossibilità di verificare la sopravvivenza dei token durante la generazione ha reso l'approccio diretto autoregressivo (AR) con VQ-GAN di fatto inutilizzabile: senza un riscontro locale sulla coerenza tra il token selezionato e quello che il destinatario osserverà dopo il round-trip attraverso il canale pixel, la decodifica steganografica produce un flusso di bit corrotto già dopo i primi token. Il $\sim 19\%$ di token che non sopravvivono al ciclo encoding-decoding-encoding si traduce in un tasso d'errore sul messaggio recuperato che rende impossibile qualsiasi comunicazione affidabile anche utilizzando algoritmi di correzione d'errore sui bit del messaggio. In altre parole, il canale visivo così costruito non è *lossless* e non può supportare la trasmissione di informazione segreta in modo affidabile.

La causa profonda è la non-invertibilità intrinseca dell'architettura: l'encoder e il decoder sono funzioni statistiche apprese separatamente, non coppie di trasformazioni inverse progettate per essere biettive. La quantizzazione vettoriale introduce un'ulteriore perdita: due vettori latenti simili possono essere quantizzati allo stesso indice, ma la mappa inversa non è garantita. A differenza di un LLM, dove il token generato è esattamente il token che il destinatario legge (il canale testuale è *lossless*), nella VQ-GAN il token scelto dal Transformer attraversa un canale rumoroso (il passaggio attraverso pixel continui e la riquantizzazione dell'encoder), ovvero soggetto ad errori sistematici, che ne altera l'identità in quasi metà dei casi.

3.2 TENTATIVO DI FINE-TUNING DELL'ENCODER VQ-GAN

3.2.1 Obiettivo

Per superare la limitazione della non-invertibilità, abbiamo esplorato la possibilità di addestrare l'encoder VQ-GAN a produrre codici invertibili rispetto al decoder fisso, tramite un fine-tuning mirato.

3.2.2 Strategia di fine-tuning

L'approccio è stato quello di congelare i pesi del decoder e del codebook della VQ-GAN — componenti che contribuiscono direttamente alla generazione delle immagini e la cui modifica avrebbe alterato l'inferenza

— e addestrare soltanto l’encoder con una funzione di loss modificata. L’obiettivo era minimizzare l’errore di ricostruzione del codice:

$$\mathcal{L}_{\text{inv}} = \|\mathbf{S} - \text{Encoder}_{\text{VQ}}(\text{Decoder}_{\text{VQ}}(\mathbf{S}))\|_2^2,$$

dove $\mathbf{S}$ è la griglia di indici discreti originali e $\text{Encoder}_{\text{VQ}}(\text{Decoder}_{\text{VQ}}(\mathbf{S}))$ è la griglia ottenuta ricodificando l’immagine decodificata. Minimizzando questa loss, l’encoder è incoraggiato a produrre rappresentazioni latenti che, dopo decodifica e ricodifica, preservino gli indici originali.

3.2.3 Risultati

Il fine-tuning ha prodotto un miglioramento misurabile: l’errore di ricostruzione dei codici si è ridotto significativamente, con un aumento della percentuale di token invariati dopo il round-trip encoding-decoding-encoding fino al ~98%. Tuttavia, anche dopo l’addestramento, una frazione non trascurabile di token continuava a subire fluttuazioni. Operando questa analisi, abbiamo notato che in realtà il *flip-rate* precedente non risultava pari al 19% come stimato inizialmente, ma era più basso, circa il 10%. Questo perché la VQ-GAN non è un canale binario indipendente per ciascun token: la dipendenza tra i token e la propagazione dell’errore attraverso il decoder convoluzionale produce un effetto “a cascata” che amplifica gli errori locali. A questo si aggiunge il problema della compressione in PNG a 8 bit per colore, che introduce una variazione dei pixel, dovuta alla compressione e alla quantizzazione, la quale altera la ricodifica dei token.

L’analisi ha mostrato che il condizionamento intrinseco della VQ-GAN, ovvero la dipendenza di ciascuna posizione (r, c) dall’intera griglia di token circostanti, impedisce una convergenza a zero dell’errore. Modificare un singolo token nella griglia altera la ricostruzione dell’intera immagine, che a sua volta altera la codifica di tutte le altre posizioni. Questo effetto di “propagazione laterale” rende il problema intrattabile attraverso un fine-tuning dell’encoder.

3.3 EQUIVALENZA DEL TRANSFORMER: METEOR È APPLICABILE AI TOKEN VISIVI

3.3.1 La verifica concettuale

Pur con le limitazioni legate alla non-invertibilità, abbiamo voluto verificare se, a livello di Transformer, l’algoritmo di campionamento di Meteor

fosse applicabile anche ai token visivi. Per rispondere, abbiamo isolato il Transformer dal tokenizer e confrontato i due processi di generazione. In Meteor, un LLM produce a ogni passo una distribuzione $P(w_t | w_{<t})$ sul vocabolario testuale; l'arithmetic coding consuma una porzione del messaggio segreto e seleziona il token w_t il cui intervallo cumulativo contiene i bit letti. Nel Transformer della VQ-GAN, il modello produce una distribuzione $P(s_t | s_{<t})$ sul codebook e l'arithmetic coding seleziona l'indice s_t con lo stesso identico criterio. Le due operazioni sono formalmente equivalenti: in entrambi i casi si seleziona un elemento da un insieme finito secondo una distribuzione di probabilità, guidati da una sequenza binaria. La conclusione è che l'algoritmo di Meteor è direttamente trasponibile ai token visivi; il vero ostacolo non risiede nel meccanismo di campionamento, ma nella fedeltà del canale encoding-decoding, cioè nella capacità del tokenizer di restituire, dopo il round-trip attraverso i pixel, gli stessi indici su cui il Transformer è stato condizionato. A conferma di ciò, si è verificato sperimentalmente che l'implementazione senza passare dal dominio dei pixel ha ottenuto il 100% di successo nel ripristinare il messaggio originale.

3.3.2 Implicazione per la scelta dell'architettura

Questa verifica ha orientato la ricerca verso un tokenizer visivo che offrisse garanzie di invertibilità. L'architettura candidata è stata **Open-MAGVIT2** [luo-2025], che utilizza la *Lookup-Free Quantization (LFQ)*, una tecnica di quantizzazione intrinsecamente invertibile ([sottosottosezione 2.3.3](#)).

3.4 SPERIMENTAZIONE CON OPEN-MAGVIT2

3.4.1 Motivazione

Open-MAGVIT2 presenta tre caratteristiche che lo rendono particolarmente adatto al nostro scopo:

1. **LFQ invertibile:** la quantizzazione per segno di ciascuna dimensione latente produce un codice binario deterministico e invertibile. Dato un codice $\mathbf{z}_q \in \{-1, 1\}^d$, la decodifica produce un'immagine, e ricodificando tale immagine si ottiene lo stesso codice (a meno del rumore del canale PNG).

2. **Modello autoregressivo:** il *Net2NetTransformer* associato a Open-MAGVIT2 genera token visivi in modo autoregressivo, con una distribuzione di probabilità per ciascuna posizione.
3. **Fattorizzazione asimmetrica:** la suddivisione in pre-token (6 bit) e post-token (12 bit) consente di operare a granularità variabile.

3.4.2 Il problema della propagazione dell'errore

Nonostante l'invertibilità della LFQ, la sperimentazione con Open-MAGVIT2 ha rivelato un secondo problema fondamentale: la *propagazione dell'errore* nel modello autoregressivo.

Il meccanismo è il seguente:

1. L'encoder steganografico genera sequenzialmente i token visivi, dove ciascun token s_t è determinato dai bit del messaggio e dalla distribuzione $P(s_t | s_{<t})$ prodotta dal Transformer.
2. L'immagine generata viene convertita in PNG e trasmessa.
3. Il destinatario riceve l'immagine, la riconverte in token, e tenta di estrarre il messaggio. La decodifica richiede di ripercorrere la sequenza: per ciascuna posizione t , il destinatario deve ricostruire la distribuzione $P(s_t | s_{<t})$ usando i token $s_{<t}$ **così come appaiono nell'immagine ricevuta**.

Il punto critico è che i token ricevuti $s'_{<t}$ possono differire da quelli originali $s_{<t}$ a causa del rumore del canale PNG. Se un singolo token s_k (per qualche $k < t$) viene alterato, tutte le distribuzioni successive $P(s_t | s'_{<t})$ sono calcolate su un contesto corrotto e non corrispondono più a quelle utilizzate dal mittente. Questo genera una cascata: un errore in posizione k corrompe tutte le predizioni da $k + 1$ in poi.

A questo si aggiunge un secondo effetto: l'arithmetic coding a bitrate variabile. Poiché token diversi possono codificare un numero diverso di bit (in funzione dell'entropia della distribuzione), un singolo flip di token altera il numero di bit che il destinatario legge in quella posizione, causando un disallineamento dell'intero flusso binario — un secondo meccanismo indipendente di desincronizzazione.

3.4.3 Misurazione dell'errore

Il canale di trasmissione considerato nella sperimentazione è il seguente: una griglia di token viene convertita in un'immagine PNG a 8 bit (formato lossless, ma quantizzato a uint8), trasmessa, ricaricata e riconvertita in token. La domanda cruciale è quanto fedelmente i token sopravvivano a questo round-trip.

Le misurazioni empiriche sul modello reale hanno caratterizzato il rumore del canale:

- **Token flip rate** $\approx 16.8\%$: circa un token su sei viene alterato.
- **Per-bit flip rate** $\approx 1.1\%$: la probabilità che un singolo bit di una dimensione latente venga alterato nel passaggio attraverso il canale è circa l'1%.

La relazione tra i due valori segue la formula $1-(1-0.011)^{18} \approx 0.18$: un token subisce un flip se *almeno uno* dei suoi 18 bit viene alterato. Questo divario (1% a livello di bit, 18% a livello di token) è fondamentale per comprendere le strategie di embedding descritte in seguito: uno schema che tratta il token come simbolo atomico vede un canale con il 18% di rumore; uno schema che opera a livello di bit vede un canale con solo l'1% di rumore.

Su questo canale, abbiamo misurato l'errore end-to-end del sistema con Open-MAGVIT2. I risultati sono stati:

- **Bit error rate (BER)** $\approx 65\%$ nella configurazione base di arithmetic coding.
- Il BER mostrava una tendenza a *crescere* (verso 80–84%) quando veniva introdotta una simulazione lato mittente per tentare di allineare il contesto, indicando l'assenza di un punto fisso nel ciclo di feedback.

La ragione per cui non esiste punto fisso è la circolarità del problema: per allineare il contesto del mittente con quello del destinatario, il mittente dovrebbe condizionarsi sui token ricodificati s' ; ma i token scelti dal mittente sono quelli che vengono renderizzati, e renderizzarli produce un s' *diverso*. La circolarità è irriducibile.

Tentativo: binning a rate fisso

Un tentativo di mitigazione è stato il passaggio da arithmetic coding a **fixed-rate binning**: invece di lasciare che il numero di bit per token dipendesse dall'entropia, abbiamo forzato esattamente b bit per sotto-token, selezionando tra i top 2^b token della distribuzione ordinata per probabilità. Questo elimina il problema del disallineamento del flusso binario, ma non risolve la cascata: il *ranking* dei token dipende ancora dalla distribuzione condizionata al contesto, che rimane corrotto dal primo errore.

3.4.4 *Verifica: embedding nei piani di bit*

Per confermare che il problema fosse effettivamente nella propagazione dell'errore e non nell'architettura di Open-MAGVIT2, abbiamo implementato una strategia di embedding alternativa: invece di guidare il campionamento del Transformer, abbiamo codificato i bit del messaggio *direttamente nei singoli piani di bit dei codici LFQ* di un'immagine esistente. Questo approccio elimina completamente la dipendenza dal contesto autoregressivo, poiché ciascuna posizione viene decodificata indipendentemente dalle altre.

QIM su codici LFQ

La tecnica, riconducibile alla *Quantization Index Modulation (QIM)*, funziona come segue:

Data un'immagine x , la si codifica con l'encoder LFQ per ottenere la griglia di codici $Z \in \{-1, 1\}^{h \times w \times d}$, dove $d = 18$ è il numero di dimensioni latenti per posizione. Si seleziona quindi un sottoinsieme di p piani di bit (dimensioni latenti) tra i d disponibili; su ciascun piano si sovrascrive il bit di ciascuna posizione con un bit del messaggio, opportunamente codificato con codici di correzione d'errore. Si decodifica poi la griglia modificata per ottenere l'immagine stego. Il destinatario, ricodificando l'immagine, legge i piani di bit selezionati e recupera il messaggio.

Poiché la decodifica di ciascuna posizione non dipende dalle altre, non esiste alcuna cascata: un errore in una posizione rimane confinato a quella posizione. Il rumore effettivo sul canale è quindi governato dal **per-bit flip rate** ($\approx 1.1\%$) anziché dal **token flip rate** ($\approx 16.8\%$), riducendo di oltre un ordine di grandezza la probabilità di errore sull'unità informativa.

Risultati dell'embedding nei piani di bit

L'esperimento ha prodotto risultati positivi:

- Con l'impiego di codici di correzione d'errore (Reed-Solomon su GF(256) e ripetizione binaria), è stato possibile recuperare il messaggio con **100% di affidabilità** anche dopo il passaggio attraverso il canale PNG.
- La capacità utile si attestava nell'ordine di **30–50 byte per immagine 256×256** con strength 9 (9 piani di bit su 18 utilizzati).
- Con l'utilizzo di immagini reali come carrier (invece di campioni generati dal modello autoregressivo (AR)), la capacità scala linearmente con la risoluzione: un'immagine 512×512 ha 4 volte i token di una 256×256 , offrendo 4 volte la capacità a parità di strength.

3.4.5 *Tensione fondamentale: robustezza vs impercettibilità*

L'esperimento di embedding nei piani di bit ha anche messo in luce una tensione fondamentale, che rappresenta il limite intrinseco di questo approccio.

Un bit di una dimensione latente è *robusto* (sopravvive al canale PNG) quando la dimensione corrispondente ha *magnitudo elevata*, ovvero lontana dallo zero, dove il rumore del canale difficilmente ne inverte il segno. Tuttavia, forzare il segno di una dimensione a magnitudo elevata produce una *grande perturbazione nell'immagine*. Al contrario, le dimensioni a magnitudo ridotta possono essere modificate con minor impatto visivo, ma sono più vulnerabili al rumore del canale.

Le misurazioni sui vari piani di bit mostrano una variabilità significativa: i piani più robusti hanno un flip rate del $\sim 2\%$, quelli più fragili arrivano a $\sim 11\%$. Inoltre, sovrascrivere più di circa metà dei 18 piani di bit (strength > 9) spinge l'immagine fuori dal manifold naturale del tokenizer, degradando tutti i piani e portando il flip rate effettivo dal $\sim 4\%$ al 11–18%.

Questa tensione è la manifestazione, in una nuova forma, dello stesso problema incontrato con Meteor/autoregressivo: *l'indistinguibilità statistica e la robustezza rispetto a un canale rumoroso sono mutuamente esclusive* per questo tipo di sistemi. Lo schema basato sulla scelta del token (autoregressivo) acquista indistinguibilità e paga con la fragilità; lo schema basato sui piani di bit (QIM) acquista robustezza e paga con la rilevabilità.

3.5 SINTESI DEI RISULTATI SPERIMENTALI

La tabella seguente riassume i risultati ottenuti nei vari esperimenti:

Configurazione	Canale	BER	Recupero
VQ-GAN + Meteor	Pixel PNG	~ 19%	Parziale
VQ-GAN + fine-tuning encoder	Pixel PNG	~ 10%	Parziale
OpenMAGVIT2 + Meteor	Pixel PNG	~ 65%	Assente
OpenMAGVIT2 + Meteor	Token lossless	0%	Completo
OpenMAGVIT2 + QIM	Pixel PNG	< 5%	Completo

Tabella 1: Sintesi dei risultati sperimentali

I risultati confermano che:

1. L'algoritmo di campionamento di Meteor è concettualmente applicabile ai token visivi, come dimostrato dall'esperimento su canale lossless (recupero esatto del 100% dei bit).
2. La *non-invertibilità* della VQ-GAN impedisce un'applicazione diretta, e il fine-tuning dell'encoder non è sufficiente a risolvere strutturalmente il problema.
3. Anche con un tokenizer invertibile (Open-MAGVIT2/LFQ), la *propagazione dell'errore* nel modello autoregressivo rende il recupero impossibile su un canale pixel rumoroso.
4. L'*embedding diretto nei piani di bit* (QIM) supera il problema della propagazione, ma al costo di essere un watermark rilevabile, non una steganografia indistinguibile, e quindi fortemente suscettibile ad algoritmi di steganalisi.

3.6 LIMITAZIONI EMERSE

La sperimentazione ha evidenziato le seguenti limitazioni fondamentali:

- **Indistinguibilità e robustezza sono incompatibili** per questo tipo di sistemi: un metodo che opera sulle scelte del modello autoregressivo è statisticamente indistinguibile ma fragile; un metodo che opera direttamente sui codici (QIM) è robusto ma rilevabile.

- **Modesta capacità utile:** anche nello scenario più favorevole (QIM su 9 piani di bit), la capacità è dell'ordine di poche decine di byte per immagine 256×256 . Per applicazioni pratiche è necessario scalare la risoluzione del carrier o suddividere il payload su più immagini.
- **Dipendenza dal canale:** tutte le calibrazioni sono state effettuate per il canale PNG/uint8. Canali più ostili (JPEG, resize, crop) richiederebbero una ricalibrazione e produrrebbero capacità inferiori.
- **Qualità dell'immagine vs strength:** la fedeltà visiva dell'immagine carrier è inversamente proporzionale al numero di piani di bit sovrascritti. Carrier riconoscibili richiedono strength bassa, e quindi capacità ridotta.

CONCLUSIONI

4.1 RIEPILOGO DEI RISULTATI

Il presente lavoro ha investigato l'applicabilità del paradigma di *Steganography without Embedding* al dominio visivo, utilizzando come riferimento il framework Meteor per il dominio testuale e due architetture di tokenizzazione visiva: VQ-GAN e Open-MAGVIT2.

Il percorso sperimentale ha permesso di identificare e isolare due problemi fondamentali:

1. **Non-invertibilità del tokenizer:** l'architettura VQ-GAN, pur essendo strutturalmente analoga a un modello linguistico (codebook discreto + Transformer autoregressivo), non garantisce l'invertibilità del processo di encoding/decoding. L'encoder e il decoder sono funzioni statistiche apprese separatamente, non coppie di trasformazioni inverse. Il tentativo di fine tuning dell'encoder ha ridotto ma non eliminato l'errore, a causa del condizionamento laterale tra posizioni adiacenti nella griglia.
2. **Propagazione dell'errore nel modello autoregressivo:** anche utilizzando un tokenizer intrinsecamente invertibile (Open-MAGVIT2 con LFQ), la natura autoregressiva del modello di generazione produce una cascata di errori: un singolo token alterato dal rumore del canale PNG corrompe tutte le predizioni successive, rendendo il recupero del messaggio impossibile.

La verifica dell'equivalenza a livello di Transformer ha confermato che l'algoritmo di campionamento di Meteor è formalmente identico per token testuali e visivi, quindi il problema non è nell'algoritmo ma nella fedeltà del canale di comunicazione. L'esperimento su canale lossless (token diretto, senza passaggio attraverso pixel) ha dimostrato un recupero esatto del 100% dei bit, confermando la validità concettuale

dell'approccio.

L'embedding diretto nei piani di bit (QIM) ha dimostrato che, eliminando la dipendenza dal contesto autoregressivo, il canale visivo può supportare la comunicazione steganografica con recupero affidabile ($< 5\%$ BER, corretto da Reed-Solomon). Tuttavia, questo approccio costituisce un watermark rilevabile, non una steganografia statisticamente indistinguibile.

4.2 CONTRIBUTI

I contributi principali di questa tesi sono:

- L'identificazione e la caratterizzazione della **non-invertibilità della VQ-GAN** come fattore limitante per l'applicazione di SwE al dominio visivo, con analisi del tentativo di fine tuning e delle sue limitazioni.
- La dimostrazione dell'**equivalenza formale tra Meteor testuale e Meteor visivo** a livello di Transformer, che consente di separare il problema dell'algoritmo di campionamento dal problema del canale di comunicazione.
- L'identificazione della **propagazione dell'errore** come ostacolo fondamentale per sistemi SwE basati su modelli autoregressivi, con analisi del meccanismo di cascata e delle sue conseguenze.
- La validazione dell'**embedding diretto nei piani di bit della LFQ** come tecnica alternativa, dimostrando che Open-MAGVIT2 può supportare watermarking robusto su canale PNG, e la caratterizzazione della tensione fondamentale tra robustezza e impercettibilità.

4.3 LAVORI FUTURI

Sulla base dei risultati ottenuti, si possono delineare diverse direzioni per la ricerca futura:

- **Sistemi ibridi AR-QIM**: una direzione promettente consiste nell'integrare il campionamento autoregressivo con l'embedding diretto nei piani di bit. La maggior parte dei token verrebbe generata tramite arithmetic coding, preservando l'indistinguibilità statistica del carrier, mentre un sottoinsieme di piani di bit sarebbe riservato

a codici di correzione d'errore o checksum per la verifica locale dell'integrità. Un tale schema ibrido potrebbe offrire un compromesso pragmatico: robustezza sufficiente a sopravvivere al canale PNG senza sacrificare completamente la natura steganografica del sistema.

- **Caratterizzazione di canali lossy:** l'analisi del canale PNG/uint8 ha rivelato un flip rate per-bit sorprendentemente basso ($\sim 1.1\%$), ma questa caratterizzazione è specifica per la compressione lossless. Estendere lo studio a canali effettivamente lossy (JPEG, WebP, ridimensionamento, ritaglio) consentirebbe di quantificare la sopravvivenza dei codici LFQ sotto trasformazioni aggressive e di progettare schemi di embedding con garanzie di recupero tarate sul canale atteso. In particolare, comprendere come il rumore di compressione si distribuisca tra i piani di bit permetterebbe di allocare il payload sui piani più robusti, massimizzando la capacità residua.
- **Ricerca con pruning dei percorsi di campionamento:** per mitigare la propagazione dell'errore senza rinunciare al paradigma autoregressivo, si potrebbe adottare un algoritmo di ricerca su grafo con potatura (*beam search*). Invece di selezionare deterministicamente un singolo token a ogni passo, il sistema manterrebbe un fascio di ipotesi sulle possibili sequenze di token candidate, calcolando per ciascuna la probabilità condizionata dato il contesto che il destinatario osserverebbe dopo il round-trip attraverso il canale pixel. I percorsi la cui probabilità cumulativa scende sotto una soglia verrebbero potati; il percorso sopravvissuto rappresenterebbe la sequenza più verosimilmente corretta per la ricostruzione del messaggio. Questo approccio richiederebbe di simulare il canale a ogni passo di decodifica, con un costo computazionale significativo.
- **Codici convolutivi per l'allineamento sequenziale:** i codici di correzione d'errore convolutivi offrono una proprietà particolarmente adatta al problema della cascata: essi garantiscono l'integrità del messaggio in modo sequenziale, consentendo al decoder di verificare incrementalmente che tutti i token decodificati fino a quel momento siano corretti. A differenza dei codici a blocco come Reed-Solomon, che operano su finestre di dimensione fissa e non forniscono informazione sullo stato corrente della decodifica, un codice convolutivo permetterebbe di rilevare immediatamente l'insorgere di un errore e di reinizializzare il contesto autoregressivo,

interrompendo la propagazione prima che corrompa l'intero flusso di bit. La sfida principale è l'integrazione con l'arithmetic coding a bitrate variabile, che richiederebbe un adattamento dello schema di codifica.

- **Quantizzazione invertibile con robustezza al canale:** la LFQ di Open-MAGVIT2 garantisce l'invertibilità matematica del round-trip encoding-decoding (ogni vettore latente è univocamente determinato dal segno delle sue componenti), ma non offre alcuna protezione contro il rumore introdotto dal canale di trasmissione. Una direzione di ricerca rilevante è lo sviluppo di schemi di quantizzazione che siano simultaneamente invertibili e robusti: ad esempio, quantizzatori che introducono ridondanza controllata nello spazio latente, o tecniche di *pre-emphasis* che amplificano selettivamente le componenti più fragili del codice prima della renderizzazione in pixel, per compensare l'attenuazione attesa dal canale. L'obiettivo è preservare la biunivocità della rappresentazione latente anche dopo il passaggio attraverso un canale con perdita, condizione necessaria per realizzare uno SwE visivo pienamente funzionante.
- **Codifica a ripetizione con voto a maggioranza su piani a bassa magnitudo:** la tensione tra robustezza e impercettibilità discussa nel [Capitolo 3](#) potrebbe essere attenuata sfruttando il fatto che le dimensioni latenti a magnitudo ridotta sono quasi invisibili se alterate, ma individualmente fragili. L'idea è codificare ciascun bit del messaggio su un gruppo di n di queste dimensioni, anziché su una singola dimensione a magnitudo elevata. In decodifica, si osserva il segno di ciascuna delle n dimensioni e si decide il bit per **voto a maggioranza**: se la maggior parte di esse ha mantenuto il segno corretto, il bit è recuperato. La probabilità che il voto sia sbagliato, cioè che più della metà delle n dimensioni subiscano un flip, è data dalla coda di una distribuzione binomiale e decade rapidamente all'aumentare di n . Con $n = 7$ e un flip rate individuale dell'11% (tipico per dimensioni a bassa magnitudo), la probabilità d'errore scende a circa lo 0.2%, paragonabile a quella di una dimensione robusta, ma con un'alterazione dell'immagine molto più contenuta perché distribuita su molte componenti di piccolo impatto. La perdita di capacità (un fattore $1/n$) può essere compensata aumentando il numero di piani utilizzati, poiché il danno visivo cumulativo di molte dimensioni a bassa magnitudo rimane comunque inferiore a quello di poche dimensioni a magnitudo elevata. Lo sviluppo di

questa tecnica richiederebbe una mappatura precisa del flip rate in funzione della magnitudo, per determinare la coppia (n, p_f) ottimale e, potenzialmente, un adattamento dinamico della ridondanza al contenuto dell'immagine carrier.

4.4 CONSIDERAZIONI FINALI

Il lavoro presentato in questa tesi dimostra che la trasposizione del paradigma SwE dal dominio testuale a quello visivo è **concettualmente possibile ma praticamente vincolata** da limitazioni strutturali dei tokenizer e dei modelli autoregressivi attuali. La principale barriera non è algoritmica (l'arithmetic coding funziona correttamente sui token visivi) ma ingegneristica: la mancanza di un canale di comunicazione lossless tra il mondo dei token e il mondo dei pixel.

L'embedding diretto nei piani di bit offre una soluzione pragmatica che sacrifica l'indistinguibilità in favore della robustezza, posizionandosi come watermark piuttosto che come steganografia. Il confine tra queste due discipline è proprio il punto in cui la presente sperimentazione si è arrestata, e dove la ricerca futura potrà raccogliere la sfida.